Technical reference manual

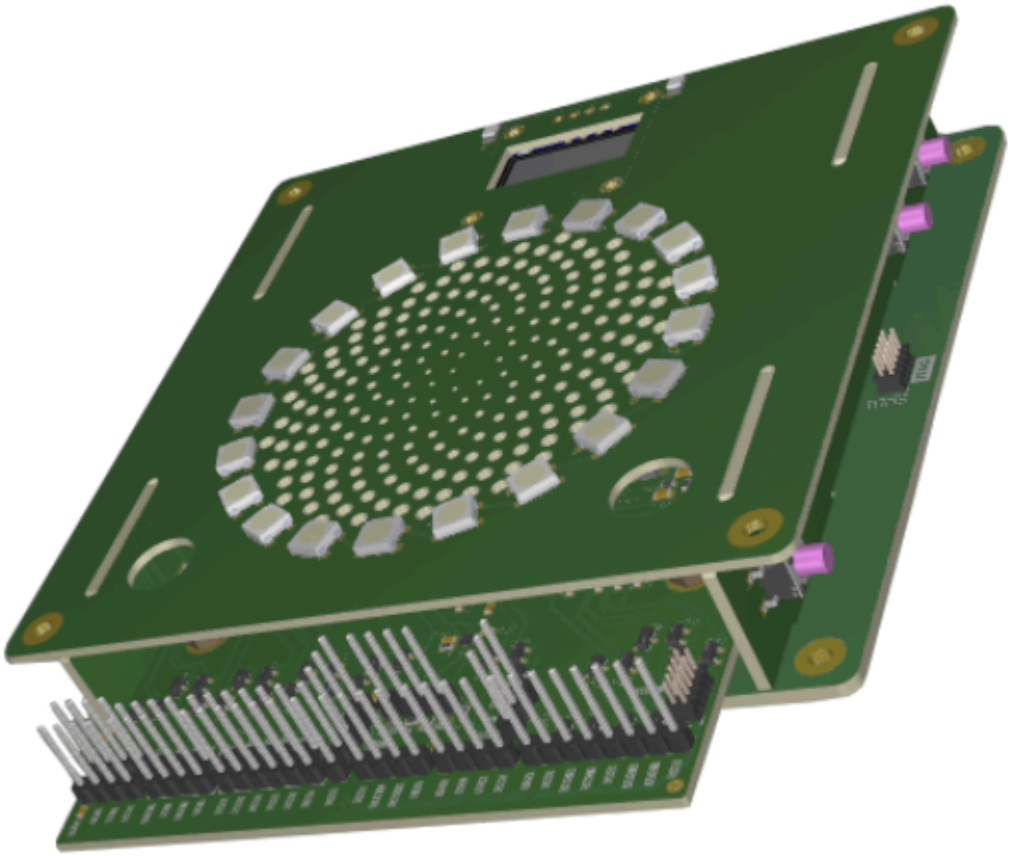
1.0.0

EtherBench

EtherBench Project

What's EtherBench ?

Etherbench is a platform, built arround a lot of different projects, designed to make debugging and testing easier, without requiring costly tools.



While it provides basic debugger features, according to the ARM CMSIS-DAP specification, over the two standard buses:

- SWD : Serial Wire Debug, common for MCUs
- JTAG : Joint Test Action Group, common for FPGAs and MPUs.

It also provides standard buses, to perform most of the IO a chip can do, within a single device. This include:

- 2 x USART buses, up to 20 Mbaud. One of them include the modem features (RTS and CTS signals).
- 1 x SPI bus, up to 50 Mbaud.
- 1 x I2C bus, up to 4 MHz.
- 1x CAN bus

And, some very basic analog features:

- 2 x Analog inputs, up to 16 bits (emulated).
- 1 x Analog Output.

All of this stuff is isolated from the host, thus even in case of failure on the DUT side, your PC remains safe.

To interract with the device, multiple options are offered:

- 100 Mbps Ethernet port (including DHCP)
- 12 Mbps USB port (behind a 480 Mbps hub, to wire others devices !).

Both options are similar in terms of features.

Highlights

All of those aspects are detailed within their associated pages. Here a rapid tour of the horizon:

- High performance hardware
  - STM32H5 MCU, providing all the features of the probe
  - Isolated PCB, ensuring safety of your designs.
- Broad range of communications
  - Ethernet
  - USB
  - SDMMC
- Easy to get hands on
  - Comprehensive leds signals
  - Standard interfaces

More infos ?

There's a lot that isn't documented here. Please check all the subsections, or download the PDF document !

- Installing the device
- Hardware schematic and details
- Programming guides
- Examples
- Software architecture

Project

Why ?

I've built this project, because I was tired to accumulating probes on my desk. One for the ST's, one for the FPGA, and so on. Therefore, I've looked into different projects, such as this one. But, it wasn't complete, and didn't like the complete implementation, so, I've just rewrote my own implementation, with the latest features available.

The project then started as, and what if I add a USB-USART bridge to it ? And, wait, can't I also add an Ethernet port ? Now, we're here, reading the project documentation...

Table of Contents

Assembling

As the device is composed of multiple boards, some tools will be required:

- A PH0 screwdriver
- A soldering setup

> **Soldering**
>
> While the soldering can be done by anyone, stay safe when doing it (airy place, free space and heat resistant tools).

Top board

The top board is the one which require soldering. Place the two side boards on the top level board notch, and solder them in place.

> **Assembly**
>
> Ensure the boards are soldered at right angle. Failing to do it will eventually end up in the inability to assemble the system.

> **Pinout**
>
> Most of the pins are ground. Therefore, it may not be surprising to get a single solder blob over all the pins. This is not an issue. Only a single connection, on side side has active signals on it.

Motherboard

Now, place the daughter board on it's dedicated slot on the motherboard, and screw it in place.

Finally, you can place the top board on the notch on the motherboard. The pogo pins on the motherboard shall make contact with the pads under the top board. Fix that in place using the four screw, on each corner. Tight them correctly, as they're going to be used for ground.

Replacing the daughter board

The two screws on the daughter board can be removed without removing the top board, as there's two 10mm hole on the right position. You can just unscrew and pull the daughter board off its slot directly.

Installing

As the project is based over three software components, installing can't be done with a single installer. Only the Firmware is required for the device to be working, all other are add-ons.

Embedded Firmware

1. You'll need a copy of the embedded firmware. The later one can be obtained from two sources:
   - Downloading a built firmware, on the releases.
   - Building your own, with the STM32Cube environnement.

   For most user, I recommend the first option.
2. Then, we need to program the chip on the board. Any tool can be used, such as CubeProgrammer, which can use the USB link to upload the firmware.

Alternatively, there's SWD pins available, thus, any USB-SWD probes can be reused.

1. Once done, the board shall come alive. If that's the case, you'r done for the firmware.

Desktop Application

1. You'll need the installer, for the right exploitation system. You can download them on the releases. pages.
2. Run the installer.
3. Done

Alternatively, you can build the software from sources, over the repo.

Software library

The software library is actually a single header file, that provide abilities for the tool to send data in a known format to the App. This expand the abilities of the App without costing that much.

There's no installation, as you copy that file into your project, include it, and build with your usual toolchain. That's all !

Configuring

Hardware

EtherBench, at least the probe is designed around an assembly of three PCB :

- One master board, with six layers. It host all the complex functions of the device.
- One top board, with the leds and basic leds features.
- Two lateral boards, with buttons.

The two later ones are done with a two layer boards, as there isn't any high speed signals nor complex requirements. Therefore, the cost of the device remains accessible.

The electrical part fit within a 110 x 110 mm boards assembly.

Master board

The master board is designed around

> **Unfinished spec**
>
> This document present reflexions about non finished designs. Therefore, details may be changed without notice.

Top board

The top board act as a user interface more than a functionnality board. it comports:

- A ring led, as the main interface with the user
- Some status leds
- An I2C screen

Led ring

The led rings, based over 20 WS2812 leds show the user animations, and statuses. This replace a complex gui with simpler animations. The GUI is then sent back over buses to the host computer.

Multiple animations can be shown :

Animation	Color	Description	Meaning
Cylon / Radar Spin	`#22F2D6`	A single bright Blue pixel spins around the 10-LED circle rapidly, leaving a fading trail of 3 pixels behind it.	Power on until RTOS threads are fully initialized and IP address is acquired.

Breathing	`#FDFF91`	All 10 LEDs slowly pulse on and off simultaneously using a sine wave function.	RTOS is idle, no active network connections, no target board connected.
Solid Activity	`#FFBF00`	The ring show a progress status of the flash interface. Restart from 0 when in debugging session.	Software has connected and claimed the USB interface.
Blink Activity	`#C9006F`	All 10 LEDs blink on and off rapidly at 5Hz (100ms on, 100ms off). Only for the device error (not the DUT).	DUT does not respond anymore (crash ?)
Progress bar	Yellow : `#E6DC20` or Violet : `#601AED` / `#9B71F0` / `#C6B1F2`	The ring show a progress status of the sequence.	User start a sequence. Yellow for custom one, Violet if triggered from the actions buttons.
Breathing	Green : `#34BA4A` or Red : `#C4314C`	All 10 LEDs slowly pulse on and off simultaneously using a sine wave function.	Sequence has ended. Here the results.
Strobe	`#E02926`	All 10 LEDs blink on and off rapidly at 5Hz (100ms on, 100ms off). Only for the device error (not the DUT).	HardFault, Ethernet disconnect, or Storage failure.

Led

In addition of the main ring leds, there's a single led, that blink when the device is powered on. It's only used to show that the system is alive.

Screen

A small I2C screen is added on the top board, and is used to show the user basic info about the device:

- Can is remove the SD
- What's it's IP
- Is there something running currently ?
- Sequences info (what failed ?)
- Error codes, to get more specific infos (rather than a generic red flash)
- Progressions
- Custom infos, from the sequence support.

The integrated UI remains quite simple, as the device is designed to be operated remotely, over Ethernet or USB.

Side boards

The side boards are used as user inputs, and thus contain essentially buttons.

For each EtherBench probe, there's two of these boards, on each side.

On the left side, the reset button is added. It enable to send to the MCU a reset request, which will be treated once the condition permit it. This mean before resetting, the network sessions will be closed, and the debugger sessions will be closed.

On the right side, there's three buttons, referred as ACTx, from 0 to 2. These can be freely programmed by the user, to do anything they want. By default, the ACT0 and ACT1 are unused, and ACT2 is the STOP button, to stop a currently running operatio?

Go / Stop button

These buttons are used as trigger for specific, sequence related actions.

Therefore, it's possible to start or stop a sequence that can be execute from within the flash, in total autonomy.

Reset button

The button will perform an hardreset of the device. That's why it's quite "hard"

Mechanical

Now, let's show all mechanical dimensions, for the case or more advanced integrations.

Ui board

Daugther board

Power

The EtherBench project features a dynamic power management system designed to handle a wide range of input voltages and currents. Because features are structurally limited by the available source power, users must understand the operating modes before any operation.

This constraint particularly affects the Ethernet section and the ARGB LED matrix, which can consume hundreds of milliamperes. This thermal and power budget cannot be sustained when the input power source is too weak.

Power profiles

Therefore, the following matrix is applied:

Power delivery profile	USB	Ethernet	Led luminosity*	Maximal DUT Power**
`20V @ 3A`	Enabled	Enabled	100 %*	48 W
`20V @ 2.25A` / `15V @ 3A`	Enabled	Enabled	100 %*	35 W
`12V @ 3A`	Enabled	Enabled	75 %*	25 W
`9V @ 3A`	Enabled	Enabled	50 %*	18 W
`5V @ 3A`	Enabled	Enabled	33 %*	7.5 W
`5V @ 1.5A`	Enabled	Enabled	20 %*	3.5 W
`5V @ 0.5A`	Enabled	Disabled	Disabled	1 W

Led luminosity

Can be overridden in software. The effective brightness will always be the lowest value between the user configuration and the hardware profile limit.

Maximal DUT Power

Represents the theoretical maximum power budget assuming ideal efficiency. Since the PCIe slot physical current limit is capped at 4.5A (up to 12V), the highest DUT power levels can only be achieved with active buck-boost circuitry on the extension board. On the default model, the DUT power is strictly capped at 15W under 3.3V.

Legacy USB-A operation

The `5V @ 0.5A` profile represents the baseline USB 2.0 specification. This mode is typically active when using legacy USB-A to USB-C cables or standard PC ports. To preserve a reasonable power margin for the DUT and prevent host port overcurrent shutdowns, the Ethernet PHY is held under hardware reset and the ARGB LEDs are powered off.

> ⚠ **Overriding limitations**
>
> Since modern PC ports can often supply more than 500 mA without advertising it via Type-C analog signaling (or over legacy cables), a dedicated CLI command allows the user to manually override the current profile (valid for 5V profiles only). Overriding the USB power profile bypasses hardware protections and shall be done at the user's own risk.

For exigeant or high-power DUTs, an external power supply can be wired directly to the extension board to bypass USB limitations. In that case, the internal regulator can be simply turned-off (Request of a 0.0V).

Power algorithms

Internally, a dedicated power management thread monitors and configures the sub-systems at 10 Hz. The values shown in the profile matrix are deliberately conservative, assuming a worst-case simultaneous consumption on the internal 3.3V and 5.0V rails.

In real-world scenarios, the infrastructure consumption is much lower. Therefore, the internal control loop dynamically allocates the unused residual power to the DUT in real-time.

All analog measurements (from the INA232 system monitor and the DUT ADC) are filtered using a DSP filter to prevent edge-triggering or oscillation during rapid load transients. Power budgets are stable and managed linearly.

Power sequencing

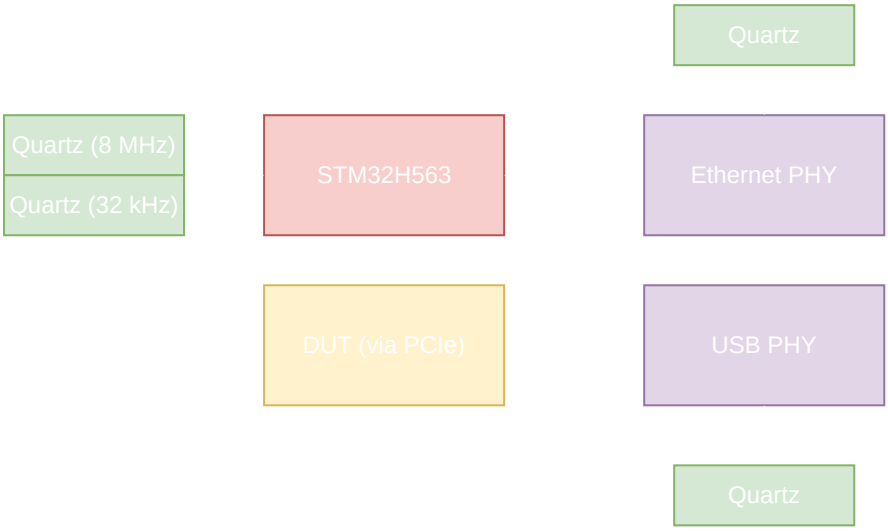
When the power is first applied (by the two CC1 and CC2 resistors on the USB-C socket), the primary, 5V regulator start it's operation once the input has reached 4.5V*. Once the first regulator as reached 92% of it's nominal voltage, the secondary 3.3V regulator start it's operation.

After the 3.3V rail is released, the STM32 MCU can start booting, and executing the startup sequence. Once done, the DUT will be identified, and parameters will be applied. The DUT dedicated regulator will then start it's operation.

Clocking

Structure

The EtherBench motherboard integrates multiple independent clock domains to ensure optimal signal integrity and flexibility. The primary system clock is derived from an external 8 MHz HSE oscillator.



The STM32H5 uses its internal Main PLL (PLL1) to scale this reference up to a 250 MHz core clock, while its Master Clock Output (MCO1) is routed to the PCIe extension connector to provide a highly configurable clock source for the DUT. Both the Ethernet PHY and USB Hub are autonomous regarding the clocks, as they each have a quartz. The Ethernet PHY provide a clock output, which is used by the Ethernet peripheral.

Clock outputs

The STM32 provide a clock output, variable for the DUT. There's two choices :

- The core clock, at 250 MHz
- The HSI oscillator, at 64 MHz.

Available rates

A hardware integer division factor (from 1 to 15) can be applied to either source. While the hardware can physically output high-frequency signals, it is strongly recommended to limit the MCO1 frequency to 64 MHz to preserve signal integrity across the connector interface.

The available output frequencies are mapped below:

Divisor	HSI64	PLL	HSI48	HSE	LSE
1	**32 MHz**	*250 MHz*	**48 MHz**	**8 Mhz**	**32.768 kHz**

2	**16 MHz**	*125 MHz*	**24 MHz**	**4 MHz**	**16.384 kHz**
3	*10.66 MHz*	*83.33 MHz*	**16 MHz**	*2.66 MHz*	*10.923 kHz*
4	**8 MHz**	*62.5 MHz*	**12 MHz**	**2 MHz**	**8.192 kHz**
5	**6.4 MHz**	**50 MHz**	**9.6 MHz**	**1.6 MHz**	*6.554 kHz*
6	*5.33 MHz*	*41.66 MHz*	**8 MHz**	*1.33 MHz*	*5.461 kHz*
7	*4.57 MHz*	*35.71 MHz*	*6.85 MHz*	*1.14 MHz*	*4.681 kHz*
8	**4 MHz**	**31.25 MHz**	**6 MHz**	**1 MHz**	**4.096 kHz**
9	*3.55 MHz*	*27.77 MHz*	*5.33 MHz*	*888 kHz*	*3.641 kHz*
10	**6.4 MHz**	**25 MHz**	**4.8 MHz**	**800 kHz**	*3.27 kHz*
11	*2.90 MHz*	*22.72 MHz*	*4.36 MHz*	*727 kHz*	*2.979 kHz*
12	*2.66 MHz*	*20.83 MHz*	**4 MHz**	*666 kHz*	*2.731 kHz*
13	*2.46 MHz*	*19.23 MHz*	*3.69 MHz*	*610 kHz*	*2.521 kHz*
14	*2.28 MHz*	*17.85 MHz*	*3.42 MHz*	*570 kHz*	*2.341 kHz*
15	*2.13 MHz*	*16.66 MHz*	**3.2 MHz**	*533 kHz*	*2.185 kHz*

Rate selection

The rates in **bold** shall be preferred over the others, as they're integer values, and therefore, more accurates.

Selection Behavior

When a specific frequency is requested via the CLI or API, the configuration algorithm automatically biases its choice toward these preferred bold values by expanding their error tolerance window to three times that of the non-standard (italic) rates.

In any cases, the selected value **will** be one of the table. Physically, it's not possible to generate any other rate, from hardware limitations. Therefore, for very specific request, the command can introduce an error. The command will return the applied frequency.

Extension boards

The whole project is based on different boards, including an extension board. The latter one target different missions:

- Adapt the signal to the required usage (voltage, isolation, current...).
- Protect the main board, in case of failure of the DUT (short-circuit, over-voltage...).
- Be a devboard by itself, in the meaning, nothing prevent (that's even an handled case !) to put the target on the devboard, and play with it !

Therefore, it must be replacable, but also physically tied to the main board to protect them against mechanical damages. To ensure that, we choosed to use the PCIe 4x connector, to provide both power and signals to this board. The form factor isn't compliant, as well as the pinout.

This connector was preferred, as it's able to use standard 1.6mm PCB, which only require fingers and an angle. This greatly reduce cost for custom boards, as it won't require any costly option. From our checks, you can get a compatible board from JLCPCB starting at 25€ (4 layers + ENIG).

Mechanical format

The board require to be in this format. The length that goes outside (L) can be extended beyond, at the condition:

- Your case can accept it
- The mechanical structure remain adequate for the load to be applied.

[TODO : Export from altium]

Electrical consideration

There's two power rails :

Rail	Value	Current	Host capacitance	Notes
MASTER_VDD	Fixed to 3.3V	Up to 0.5 A	At least 10 uF	Used to supply the onboard EEPROM and level shifters
DUT_VDD	Variable between 0.6 and VUSB	Up to 4 A	At least 100 uF	Used to supply the test logic

All signals that are going over the PCIe connector are referrenced in the MASTER_VDD domain. DUT_VDD is only applicable after the outputs are adapted by buffer, or any logic circuits.

Pinout

The PCIe pinout is inspired by the standard PCIe, but vastly differ passed the notch. In any case, **DO NOT HOOK STANDARD PCIe BOARDS** (The layout make that impossible, but with extenders...).

Here the complete pinout :

Pin	Side A	Side B
1	BOARD_PRESENT	DUT_VDD
2	DUT_VDD	DUT_VDD
3	DUT_VDD	DUT_VDD
4	GND	GND
5	TCK / SWCLK	MANAGEMENT_SCL
6	TDI	MANAGEMENT_SDA
7	TDO / SWO	GND
8	TMS / SWDIO	MASTER_VDD
9	DUT_VDD	DUT_RESET
10	DUT_VDD	GND
11	BUFFER_EN	DUT_PRESENT (DUT must pull to DUT_VDD)
NOTCH	**NOTCH**	**NOTCH**
12	I3C_SDA	I3C_SCL
13	CLK	GND
14	GND	USB DP
15	GPIO PUSH_PULL 0	USB DM
16	DAC 0	GPIO OPEN_DRAIN 0

17	DAC 1	GPIO OPEN_DRAIN 1
18	GND	GND
19	GPIO PUSH_PULL 1	CAN TX
20	GND	CAN RX
21	ADC 0	GPIO OPEN_DRAIN READ 0
22	ADC 1	GPIO OPEN_DRAIN READ 1
23	SPI SS1	SPI SS0
24	GND	SPI_MOSI
25	SPI_MISO	GND
26	SPI SCLK	GND
27	USART1 RTS	USART1 RX
28	USART1 CTS	USART1 TX
29	USART2 RX	GND
30	USART2 TX	GND
31	GND	USART3 RX
32	GND	USART3 TX

Isolation

The PCIe slot isn't isolated from the DUT by default. This remains an option for specific required. In that case, an isolated power supply will also be required to convey the provided voltage to the isolated island.

The motherboard was designed with that requirement in mind, therefore only the two support screw require a carefull attention, as they're on the main ground domain. It's on the daughter board to treat them correctly.

Pinout & Hardware Mapping

The core of the EtherBench motherboard features an STM32H5 in a VFBGA201 package, exposing an extensive array of hardware peripherals. To streamline firmware development (BSP handle definition) and facilitate non-intrusive laboratory troubleshooting, all active pins are detailed below.

Testpoints (TP) are specified only for signals where trace velocity, parasitic capacitance, and hardware layout constraints allow safe physical probing.

Host communication

These signals are used as the core functions, such as host communication or flash storage.

Instance	Signal	Pin	BGA	Description
USBHS1	DP DM	PA12 PA11	B15 C15	USB peripheral, routed to the hub as a device.
USB_SUSP	USB_SUSP	PD2	D12	USB Upstream suspended signal.
USB_HS	USB_HS	PD3	D11	USB Upstream as high-speed signal.
USB_RESET	USB_RESET	PH13	E12	USB hub reset signal
UCPD1	CC1 CC2 DB_CC1 DB_CC2	PB13 PB14 PA9 PC9	P13 R14 E15 F14	USB-C signals. Routed directly to the USB-C port.
ETH	CLK MDC MDIO CRS_DV TX_EN RXD0 RXD1 TXD0 TXD1	PA1 PC1 PA2 PA7 PB11 PC4 PC5 PB12 PB15	N2 M3 P2 R3 R13 N5 P5 P12 R15	Ethernet signals, routed to the PHY.
ETH_RESET	ETH_RESET	PD9	P14	Ethernet PHY reset
OCTOSPI1	CLK IO0 IO1 IO2			

	IO3 NCS	PF10 PF8 PF9 PF7 PF6 PE11	L1 L3 L2 K1 K2 P10	QSPI Flash signals.
DEBUG	SWCLK SWDIO	PA14 PA13	A14 A15	SWD signals, routed to a standard header for initial flash.
SDMMC2	SCK CMD D0 D1 D2 D3	PD6 PD7 PG9 PG10 PG11 PG12	B11 A11 C10 B10 B9 B8	SD systems. Limited to 3.3V bus.
EVENTOUT	TRIG_OUT	PD10	N15	Trigger output.
EXTI	TRIG_IN	PD8	N15	Trigger input

Programmer

These signals are the one used as the SWD/JTAG signals to flash the probe. They're placed appart, as they're runned not used as standard peripherals.

Instance	Signal	Pin	BGA	Description
SWD / JTAG	SCLK MISO MOSI IO	PC10 - PI1 PI2 PI3 PC12	D14 - B14 C14 C13 A12	DUT Programmation signals.

Operation principle

As the SWD bus require a bi-directionnal communication, we used an SPI peripheral that can do both, here, SPI3. The SPI2 peripheral is used for JTAG, where TDI and TDO pins are used. The second SPI slave run as a full-duplex slave to ensure synchronisation of the two peripherals.

When running in SWD mode, the SPI3 peripheral can be reconfigured to a serial receiver to provide the SWO ability.

DUT communication

These signals are used by the hardware IO to the DUT, on the standard serial interfaces.

Instance	Signal	Pin	BGA	Description
MCO1	MCO1	PA8	F15	Clock output for the DUT.
I3C1	SCL SDA	PB8 PB9	A5 B4	I3C master / slave.
USART10	TX RX RTS CTS	PE3 PE2 PG14 PG13	A1 A2 A7 A8	Full featured USART bus.
USART5	TX RX	PB6 PB5	B6 A6	Rx/Tx only featured USART bus.
USART11	TX RX	PF3 PF4	J2 J3	Rx/Tx only featured USART bus.
SPI4	SCLK MISO MOSI NSS0 NSS1	PE12 PE5 PE6 PI0 PC7	R10 B2 B3 E14 G15	Full duplex SPI Master. NSS are done by GPIOs.
FDCAN1	TX RX	PE1 PE0	A3 A4	FDCAN peripheral.
ADC1	IN2 IN5	PF11 PB1	R6 R4	Analog input.
DAC1	OUT1 OUT2	PA4 PA5	N4 P4	Analog output.
GPIO	OD_W_1 OD_W_2 OD_R_1 OD_R_2 PP_1 PP_2	PI8 PI9 PI11 PH2 PI5 PI4	D2 D3 D4 C4 E4 F4	GPIO (2 pair of open drain OUT + READ + 2 push pull).
BUCK_EN	EN	PI7	C2	DUT buck enable
BUCK_INT	INT	PI6	C3	DUT buck inerrupt

DUT_RESET	RESET	PH5	J4	DUT reset signal
DUT_PRESENT	PRESENT	PH4	H4	DUT present signal

User IO

These signals are used by the hardware to show informations to the user.

Instance	Signal	Pin	BGA	Description
TIM13	aRGB_OUT	PA6	P3	aRGB output for the led ring.
TIM1 - TIM2	User leds	PE9 - PE13 - PE14 - PH9 - PA3 - PA15 - PB3 - PB10	P9 - N11 - P11 - M13 - R2 - A13 - A10 - R12	PWM Leds outputs.
GPIO_INPUT	User buttons	PH6 - PH7 - PH8	M11 - M12 - N12	User buttons
GPIO_INPUT	User reset	PD12	N13	User reset request.

Miscellaneous

These signals describe all the others pins, for a lot of different functions that may not be linked together.

Instance	Signal	Pin	BGA	Description
I2C2	SCL SDA	PF1 PF0	H3 E2	I2C for management and internal usage.
I2C4	SCL SDA ALERT	PH11 PH12 PH10	L12 K12 L13	SMBUS for the USB hub management.
USART9	TX RX IN OUT	PD15 PD14 PD13 PG2	L14 M14 M15 L15	Serial bus for the INTERCOM system.
TIM3	PWM1	PC6	H15	PWM Output for the optionnal fan.

RCC_8M	IN OUT	PH0 PH1	G1 H1	Crystal for the 8 MHz clock source.
RCC_32kHz	IN OUT	PC14 PC15	E1 F1	Crystal for the 32.768 kHz Clock source.
DEBUG	DBG_01 DBG_02 DBG_03 DBG_04 DBG_IN1 DBG_IN2 DBG_LED1 DBG_LED2	PG1 PG0 PF15 PF14 PG7 PG6 PG4 PG3	N7 M7 P7 R7 J14 J15 K14 K15	Custom debug pins for the firmware build.
PRESENT	BOARD_PRESENT	PC13	D1	Daughter board presence detection.
SCREWED	SCREWED	PE4	B1	Daughter board screwed detection.
OE	OE	PF2	H2	Buffer output enable.
POWER_OK	PGOOD	PE7	R8	Master regulator power good signal.

Triggering other devices

The motherboard is equipped with a trigger output, in the format of a standard BNC connector, matched to 50 Ohms.

The system can output short pulses (3.3V, 4 ns typically) on that connector, on specific events (matched frame, error...). The configuration is done by the user.

The rising and falling times are short (< 1 ns), and boosted by the dedicated line buffer.

How to use it

This output can be wired to any scopes or electrical device that has a trigger input, to start a measurement on a specific condition. This can grealty extend the device ability, as these measures can be very specific, and way faster than we can imagine on the device.

The tool must be configured to use external trigger input, with a threshold of 1.65V.

Thermal management

On the motherboard, only few components are sensitive to thermal stress. These are:

- Power regulators
- Main MCU

Others aren't using enough power to be concerned by thermal management.

Passive cooling

By default, only passive cooling is available on the system. All the sensitive circuits use the main board as heatsink, which provide a first cooling method.

An openning on the top level board enable the natural convection to be done, and thus, extract hot air from the system. Do not try to block it.

Forced cooling

In any case, if the passive cooling is not enough, for example, for long and power intensive tests, or, simply because the external conditions require it, an 5V PWM fan connector is available.

The latter one is controlled by the STM32 from different temperature sensors, and will output a proportionnal duty cycle.

This fan can be used to force the air cycle trough the device, and thus, cool it.

Programming

All along this chapter, you'll see how can the device be accessed, and a technical description of any feature that could be usefull when working with it. For more advanced topics about how the firmware is organized, you can check the **architecture** chapter, which explain choices and show software stack.

Accessing the device

The device can be controlled from different ways, over different mediums:

Interface	USB	Ethernet
File transfer	None*	FTP server
USART Bridge	VCOM 1	None**
Debugging	USB Debugger probe, CMSIS-DAP compatible	Remote debugger probe, CMSIS-DAP compatible
Terminal	VCOM 0	Telnet Shell
SCPI	VCOM 0	Dedicated SCPI port

> - * The file transfer can still be done by removing the SD card from the device. But, as a limitation from the filesystem we're using, files can't be copied from the flash to the SD.
> - ** This feature can be emulated by the shell terminal. Some commands can perform IO, and thus this is possible. Speed may be affected.

Sequences

Sequences are a big part of the Etherbench project. A lot of steps require some form of sequencing. Currently, there air two options to address these requirements:

- Internal sequencing
- External sequencing

The first one is executed on the device, thereby providing the fastest responses. This is also the most limited option, as only very simple structures are possible. The latter one is running on any host computer. It's response times are dependent on the IO method used, and software stacks. Any algorithm can be implemented here.Any algorithm can be implemented here

Internal sequencing

The internal sequences only support five instructions:

Instruction	Syntax	Description	Example
`execute`	`execute <command>`	Execute the passed SCPI string, as it was passed by any IO.	`execute *IDN`
`if`	`if <string>`	Evaluate the condition, and, if true, execute the code next. Will stop when an `else` or `end` is found.	`if *IDN,OK`
`else`	`else`	End an `if` condition, and only evaluate if the first condition was false.	`else`
`end`	`end`	Any condition, regardless of the initial evaluation	`end`
`print`	`print <string>`	Print the passed string to the console.	`print Hello World !`
`increment`	`increment <target>`	Increment the target variable (pass or fail). Can be used to track success or fails.	`increment pass`

Variables

The internal sequence engine does not support user variables. There air only two of these : pass and fail. For a sequence to return 0 (success), `pass > 0` and `fail = 0`.

Structures

The internal sequence engine does only support comparison to the passed string, and the latest result. This one is stored within the internal memory of the interpreter.

> ⚡ **CAUTION**
>
> The internal engine does not support any control flow loops or branch jumps. If loops are required, you must use external sequencing.

External sequencing

The external sequencing come to address all the limitations of the internal sequencing. And, the good news is that they're complementary !

The external sequencing can be done with **any** tools that support SCPI or telnets commands, thus, Python with pyvisa, LabView, and others !

For the greatest implementation, you can use the Desktop App, which include a Python interpreter, as well as a pre-intergated C++ Python library, which does all the heavy lifting for you ! Remains the pleasure of typing commands, with the probe class (`probe.send("uart", 0x42)`). You can still use the standard library of your system, thus, numpy, matplotlib, and others !

And, as announced, you can use the `probe.run("flash.seq")` to run an internal sequence ! You're getting the best of both worlds : The ideal scripting ability of Python with the performance of custom sequences !

External

Markers

All along the project, different structures are used. One of the most important is the marker stored on the daughter-board, to identify the target we're working with.

This data must be stored on an I2C EEPROM at address 0x50 on the board. The page size used is 32 bytes, to match with any reference on the market. Our is ST M24C64-RMN6TP from ST, but any reference that match theses options are good. This chip is always powered with 3.3V, even when the main supply is OFF.

Principle

The markers are based over a minimal set of information, stored within 32 bytes of data, and some optionnal pages that may store more data. Thus, the protocol is extensible.

Optionnal pages are referenced within the first page.

Page 0

This main page store the basic element, that ALL extensions board must have. The data stored within is arranged as:

Offset	Size (Bytes)	Description
`0x00`	**2**	Page 0 marker, constant to 0xEBB0
`0x02`	**2**	Page version, must be 0x1000
`0x04`	**4**	Hardware version. Interpreted as M.m.pp values.
`0x08`	**2**	Default asked voltage. Interpreted as XX mV.
`0x0A`	**2**	Maximal current requested. Interpreted as XX mA.
`0x0C`	**2**	Lowest possible voltage. Interpreted as XX mV.
`0x0E`	**2**	Highest possible voltage. Interpreted as XX mV.
`0x10`	**2**	Custom flags. See bottom definition.
`0x12`	**2**	Clock frequency. Interpreted as XX MHz.
`0x14`	**4**	Optionnal page 0
`0x18`	**4**	Optionnal page 1

`0x1C`	**4**	Config CRC

Flags

Flags are individual bits, that identify a specific option

Bit	Name	Description
`0`	Isolated board	Indicate that the board isolate it's ground relatively to the master ground.
`1`	Require clock	Indicate that the board require a clock to operate.
`2`	Required USB	Indicate that the board use the onboard USB over the PCIe slot.
`3`	Active board	Indicate that the board include active components on between the host and the IOs.
`4`	Reserved	/
`5`	Reserved	/
`6`	Non standard form factor	Indicate that the board as a non standard form factor.
`7`	Ignore mouting check	Indicate to ignore the mouting security bits. This feature must be also forced from the shell.

Optionnal pages

Optionnal pages can be passed to the structure, to extend the functions of the latter one. The structure is quite simple: All of these are designed to fit over a single four byte space.

Offset	Size	Description
`0x00`	**2**	Address of the page
`0x02`	**2**	Page type (0xEBXX)

If unused, let 0xFFFFFFFF as the page ID.

Optionnal pages

As optionnal pages, a lot can be passed to expand the abilities. Here the different structures that can be passed:

Name	Type	Description
Pages	`0xEBB1`	Store up to 8 optionnal pages pointers. A single entity of this is possible.
Peripheral config	`0xEBB2`	Store peripheral basic configuration, to ensure a direct operation. Usefull with specific active components on the board.
Board name	`0xEBB3`	Store the human name of the board
Calibration	`0xEBB4`	Store calibration values for the analog IOs
Serial Number	`0xEBB5`	Store informations about how the board itself, to identify it.
Battery emulation	`0xEBBF`	Configure the system in battery mode. A variable voltage will be provided, following the configured curves.

Battery values

The battery emulation mode enables the test of battery powered DUTs. The provided voltage will then follow an approximated curve, to simulate it. The ESR value is only here for display, as it's impossible to sample all current transients to properly model it. For precise tests, add that resistance to your daughterboard.

Page structure

Offset	Size (Bytes)	Description
`0x00`	**2**	Pages marker, constant to 0xEBBF
`0x02`	**2**	Page CRC
`0x04`	**4**	Battery tag
`0x08`	**4**	Capacity (mAh) [Q] (float)
`0x0C`	**4**	Base voltage (V) [E0] (float)
`0x10`	**4**	Drain speed factor [K] (V) (float)
`0x14`	**4**	Overvoltage value (V) [A] (float)
`0x18`	**4**	Overvoltage drain speed (V) [B] (float)
`0x1C`	**4**	ESR (Ohm) (float) (unused for simulation, only for report generation)

Battery tags

The embedded firmware include pre-configured batteries, that can be directly emulated without any user configuration. Any value that is passed on top of that will be added to the loaded one. Any unknown tag will use as base all zeroes.

Type	Tag	Capacity	Base voltage	Drain speed	Overvoltage	Overvoltage drain speed	Reco ESR
`Li-Po` / `Li-ion`	`0x10010001`	2200 mAh	3.75 V	0.0075	0.350 V	25	2-5 m

`LiFe-PO4`	`0x10010002`	3500 mAh	3.25 V	0.0020	0.150 V	70	3-4 mOhm
`CR2032`	`0x10020001`	215 mAh	3.00 V	0.2000	0.075 V	135	20 Ohm
`NiMH`	`0x10020002`	2000 mAh	1.28 V	0.0035	0.159 V	40	100 mOhm
`Alkaline`	`0x10020003`	2000 mAh	1.50 V	0.0075	0.050 V	10	100 mOhm

Yaml arguments

When using the automated page builder, available on the EtherBenchProbe repo, all of this page shall be placed under the battery tag. Then, these keys are searched:

Key	Format	Default value
tag	Integer	0
capacity	Float	1000
base_voltage	Float	3.7
drain_factor	Float	0.005
overvoltage	Float	0.5
overvoltage_drain_speed	Float	10
ESR	Float	0

The present of any key will trigger the build of the page. Missing keys are going to be filled with the default value.

Calibration values

The calibration optionnal page target application where the analog input has been adapted for any reason, and some corrections must be applied. As a reminder, the internal ADC of the ST MCU is used, therefore, for precise analog IO, a specialized tool shall be used.

Page structure

Offset	Size (Bytes)	Description
`0x00`	**2**	Pages marker, constant to 0xEBB4
`0x02`	**2**	Page CRC
`0x04`	**4**	ADC 0 Offset (float)
`0x08`	**4**	ADC 1 Offset (float)
`0x0C`	**4**	ADC Gain (float)
`0x10`	**4**	DAC 0 Offset (float)
`0x14`	**4**	DAC 1 Offset (float)
`0x18`	**4**	DAC Gain (float)
`0x1C`	**2**	Calibration year
`0x1E`	**1**	Calibration month
`0x1F`	**1**	Calibration day

Yaml arguments

When using the automated page builder, available on the EtherBenchProbe repo, all of this page shall be placed under the calibration tag. Then, these keys are searched:

Key	Format	Default value
adc0_offset	Any SI value (12mV, 1V...)	`0V`

adc1_offset	Any SI value (12mV, 1V...)	`0V`
adc_gain	Float	`1.0`
dac0_offset	Any SI value (12mV, 1V...)	`0V`
dac1_offset	Any SI value (12mV, 1V...)	`0V`
dac_gain	Float	`1.0`
year	Integer (16 bits)	`2026`
month	Integer (8 bits)	`1`
day	Integer (8 bits)	`1`

The present of any key will trigger the build of the page. Missing keys are going to be filled with the default value.

Board name

The name page enable to include a human description string to be included, thus making it easier to show. If available, it's shown on the console.

Offset	Size (Bytes)	Description
`0x00`	**2**	Pages marker, constant to 0xEBB3
`0x02`	**2**	Page CRC
`0x04`	**28**	ASCII coded name.

Yaml arguments

When using the automated page builder, available on the EtherBenchProbe repo, only the name key is required to add that page.

Key	Format	Default value
name	Any text. Only 28 first characters are used. No termination required.	`None`

Pages

Offset	Size (Bytes)	Description
`0x00`	**2**	Pages marker, constant to 0xEBB1
`0x02`	**2**	Page CRC
`0x04`	**4**	Optionnal page 0
`0x08`	**4**	Optionnal page 1
`0x0C`	**4**	Optionnal page 2
`0x10`	**4**	Optionnal page 3
`0x14`	**4**	Optionnal page 4
`0x18`	**4**	Optionnal page 5
`0x1C`	**4**	Optionnal page 6

Up to 7 pages can be added, maxing out at 8 optionnal pages in total.

Yaml arguments

This page cannot be added by the yaml file. The script will add it, if required. No user action required.

Peripherals

Offset	Size (Bytes)	Description
`0x00`	**2**	Peripheral config marker, constant to 0xEBB2
`0x02`	**2**	Page CRC
`0x04`	**2**	Enabled peripherals (see mask at the bottom)
`0x06`	**1**	GPIO Direction (4 LSB) : 1 : in; 0 : out
`0x07`	**1**	GPIO Value (4 LSB)
`0x08`	**3**	UART 0 Config
`0x0B`	**3**	UART 1 Config
`0x0E`	**3**	UART 2 Config
`0x11`	**3**	SPI Config
`0x15`	**4**	CAN Config
`0x19`	**4**	JTAG / SWD Config
`0x1D`	**3**	I2C Config

Peripherals flags

Bit	Name
`0`	UART 0
`1`	UART 1
`2`	UART 2
`3`	Reserved (constant to 0)
`4`	I3C / I2C

`5`	SPI
`6`	CAN
`7`	JTAG
`8`	Reserved (constant to 0)
`9`	GPIO 0
`10`	GPIO 1
`11`	GPIO 2
`12`	GPIO 3
`13`	Reserved (constant to 0)
`14`	Reserved (constant to 0)
`15`	Reserved (constant to 0)

UART Config

Offset	Size	Description
`0x00`	**2**	Speed : Baudrate / 100
`0x02`	**1**	Bits 0-1 : Transfer size : 7 + 2'bXX. Bits 3-4 : Stop bits number : 2'bXX (1 or 2). Bit 7 : Enable parity.

I2C Config

Offset	Size	Description
`0x00`	**1**	Speed : Baudrate / 1000
`0x01`	**1**	Bit 0 : Enable slave mode. Bit 2 : Enable 10 bit address. Bits 7-5 : Slave address [10-8]

`0x02`	**1**	Slave address [7:0]

SPI Config

Offset	Size	Description
`0x00`	**2**	Speed : Baudrate / 1000
`0x02`	**1**	Transfer size (4-32)

CAN Config

Offset	Size	Description
`0x00`	**1**	???
`0x01`	**1**	???
`0x00`	**2**	???
`0x01`	**3**	???

JTAG / SWD Config

Offset	Size	Description
`0x00`	**2**	Speed : Baudrate / 1000
`0x02`	**2**	Bit 0 : Connect under reset

Yaml arguments

When using the automated page builder, available on the EtherBenchProbe repo, each bus is configured by it's own key. Therefore, are searched:

- uart0
- uart1

- uart2
- spi
- can
- jtag or swd
- i2C

The present of any key will trigger the build of the page. Missing keys are going to be filled with the default value. For each of them, the following key are searched.

UART

Key	Format	Default value
baudrate	Integer	115200
size	Integer	8
stop bits	Integer	1
partity	Integer	0

I2C

Key	Format	Default value
baudrate	Integer	115200
slave address	Integer	0
slave mod	Integer	0

SPI

Key	Format	Default value
baudrate	Any SI value (1MHz, 16MHz)	"1MHz"
size	Integer	8

CAN

Key	Format	Default value
baudrate	Any SI value (1MHz, 16MHz)	"1MHz"
size	Integer	8

JTAG/SWD

Key	Format	Default value
baudrate	Any SI value (1MHz, 16MHz)	"1MHz"
size	Integer	8

Serial Number

The serial number page can be used when more than one board may be used, therefore the need to identify them is expressed. This page can store informations that are specific to the board, rather than the standard string which is too generic

Page structure

Offset	Size (Bytes)	Description
`0x00`	**2**	Pages marker, constant to 0xEBB5
`0x02`	**2**	Page CRC
`0x04`	**4**	Serial number (free format)
`0x08`	**20**	Manufacturer string
`0x1C`	**2**	Fabrication year
`0x1E`	**1**	Fabrication month
`0x1F`	**1**	Fabrication day

Yaml arguments

When using the automated page builder, available on the EtherBenchProbe repo, all of this page shall be placed under the serial_number tag. Then, these keys are searched:

Key	Format	Default value
serial_number	Integer (32 bits)	`0xDEADBEEF`
manufacturer string	Any char array (20 character max)	`EtherBenchLabs`
year	Integer (16 bits)	`2026`
month	Integer (8 bits)	`1`
day	Integer (8 bits)	`1`

The present of any key will trigger the build of the page. Missing keys are going to be filled with the default value.

Programming Reference Manual

Complete technical API documentation for the EtherBench platform. Includes SCPI sub-systems syntax and direct low-level serial shell interactions.

Automated actions controls

actions

- SCPI Syntax: `ACTIONS`
- SCPI Syntax: `ACTIONS?`

Fast (<100µs) Non-Blocking

Configure a new action to be performed on press of the ACTx buttons.

ARGUMENTS / RETURNS

Argument	Description
`ID`	int<1,2,3>
`ACTION`	str

Returns	Description
`STATUS`	int<0...>

ADDITIONAL INFORMATION

> **WARNING**
>
> Bount actions aren't checked, and are interpreted as any other command. Therefore, ACTx buttons may be used as reset.

BUS TRANSFER EXAMPLES

Command string: `ACTION 1,*IDN`

Device transmission: `0`

Analog Subsystem Metrology (ADC / DAC)

analog

- SCPI Syntax: `ANALOG`
- SCPI Syntax: `ANALOG?`
- Shell Hook: `analog`

Very slow Non-Blocking

Perform an IO operation over the analog subsystem. Query target the ADC, Set target the DAC.

ARGUMENTS / RETURNS

Argument	Description
`SOURCE`	integer<0,1>
`VALUE`	str<3.19V...>

Returns	Description
`STATUS`	int<0...>
`[VALUE]`	str

BUS TRANSFER EXAMPLES

Command string: `ANALOG[?] 1,[3.19V]`

Device transmission: `0[,3.28V]`

Configuration Sub-System

configuration.voltage

- SCPI Syntax: `CONFIGURATION:VOLTAGE`
- SCPI Syntax: `CONFIGURATION:VOLTAGE?`
- Shell Hook: `config`

Slow Non-Blocking

Configure the IO voltage to be used.
Query read back the current voltage.
Set configure a new voltage.

ARGUMENTS / RETURNS

Argument	Description
`[VALUE]`	str<3.3V>

Returns	Description
`STATUS`	int<0...>
`[VALUE]`	<str<2.8V>

ADDITIONAL INFORMATION

> **WARNING**
>
> The configuration range is limited by onboard EEPROM, preventing from any damages.

BUS TRANSFER EXAMPLES

Command string: `CONFIGURATION:VOLTAGE[?] [2.85V]`

Device transmission: `0,[2.78V]`

Hardware Input Outputs

hardware.can

- SCPI Syntax: `HARDWARE:CAN`
- SCPI Syntax: `HARDWARE:CAN?`
- Shell Hook: `io`

Slow Blocking

Perform an IO operation over the CAN bus. Query mean read, Set mean write

ARGUMENTS / RETURNS

Argument	Description
`[PAYLOAD]`	int(s),<0x00--0xFF>

Returns	Description
`STATUS`	int<0...>
`[PAYLOAD]`	int(s),<0x00--0xFF>

ADDITIONAL INFORMATION

> **WARNING**
>
> As this bus is asynchronous, the read command may send back a LOT of data, capped to 1024 bytes (reception memory). If speed is a requirement, consider using the raw UDP streaming.

BUS TRANSFER EXAMPLES

Command string: `HARDWARE:CAN[?] [,0x487643]`

Device transmission: `0,[0x7865456]`

hardware.i2c

- SCPI Syntax: `HARDWARE:I2C`
- SCPI Syntax: `HARDWARE:I2C?`
- Shell Hook: `io`

Slow Blocking

Perform an IO operation over the I2C bus. Query mean read, Set mean write

ARGUMENTS / RETURNS

Argument	Description
`[ADDRESS]`	int<0x00--0x3FF>
`[PAYLOAD]`	int(s),<0x00--0xFF>
`[EXTENDED]`	int<0,1>
`[SPEED]`	str<100kHz,400kHz,1MHz>

Returns	Description
`STATUS`	int<0...>
`[PAYLOAD]`	int(s),<0x00--0xFF>

ADDITIONAL INFORMATION

> **WARNING**
>
> This IO operation feature a single operation. Some required back to back operation, thus emit two IO operation.

BUS TRANSFER EXAMPLES

Command string: `HARDWARE:I2C[?] [0x78][,0x487643]`

Device transmission: `0,[0x7865456]`

hardware.spi

- SCPI Syntax: `HARDWARE:SPI`
- SCPI Syntax: `HARDWARE:SPI?`
- Shell Hook: `io`

Slow Blocking

Perform an IO operation over the SPI bus. Query mean read, Set mean write. Address is the current selected slave.

ARGUMENTS / RETURNS

Argument	Description
`[ADDRESS]`	int<1--2>
`[PAYLOAD]`	int(s),<0x00--0xFF>

Returns	Description
`STATUS`	int<0...>
`[PAYLOAD]`	int(s),<0x00--0xFF>

ADDITIONAL INFORMATION

> **WARNING**
>
> This IO operation feature a single operation. Some required back to back operation, thus emit two IO operation.

BUS TRANSFER EXAMPLES

Command string: `HARDWARE:SPI[?] 1[,0x487643]`

Device transmission: `0,[0x7865456]`

hardware.spy.can

- SCPI Syntax: `HARDWARE:SPY:CAN`
- Shell Hook: `spy`

Fast (<100µs) Non-Blocking

Configure the spy module, to look over an CAN bus. The bus is then placed as a slave, and will listen for anything.

ARGUMENTS / RETURNS

This command does not take any arguments.

Returns	Description
`STATUS`	int<0...>

ADDITIONAL INFORMATION

> **WARNING**
>
> The reception buffer is capped to 1024 bytes (reception memory). Use the UDP streaming for faster transfers.

BUS TRANSFER EXAMPLES

Command string: `HARDWARE:SPY:CAN`

Device transmission: `0`

hardware.spy.i2c

- SCPI Syntax: `HARDWARE:SPY:I2C`
- Shell Hook: `spy`

Fast (<100µs) Non-Blocking

Configure the spy module, to look over an I2C bus. The bus is then placed as a slave, and will listen for anything.

ARGUMENTS / RETURNS

Argument	Description
`[ADDRESS]`	int<0x00--0x3FF>
`[EXTENDED]`	int<0,1>

Returns	Description
`STATUS`	int<0...>

ADDITIONAL INFORMATION

> **WARNING**
>
> The reception buffer is capped to 1024 bytes (reception memory). Use the UDP streaming for faster transfers.

BUS TRANSFER EXAMPLES

Command string: `HARDWARE:SPY:I2C`

Device transmission: `0`

hardware.spy.spi

- SCPI Syntax: `HARDWARE:SPY:SPI`
- Shell Hook: `spy`

Fast (<100µs) Non-Blocking

Configure the spy module, to look over an SPI bus. The bus is then placed as a slave, and will listen for anything.

ARGUMENTS / RETURNS

Argument	Description
`[ADDRESS]`	int<1--2>

Returns	Description
`STATUS`	int<0...>

ADDITIONAL INFORMATION

> **WARNING**
>
> The reception buffer is capped to 1024 bytes (reception memory). Use the UDP streaming for faster transfers.

BUS TRANSFER EXAMPLES

Command string: `HARDWARE:SPY:SPI`

Device transmission: `0`

hardware.spy.uart

- SCPI Syntax: `HARDWARE:SPY:UART`
- Shell Hook: `spy`

Fast (<100µs) Non-Blocking

Configure the spy module, to look over an UART bus. The bus is then placed as a slave, and will listen for anything.

ARGUMENTS / RETURNS

This command does not take any arguments.

Returns	Description
`STATUS`	int<0...>

ADDITIONAL INFORMATION

> **WARNING**
>
> The reception buffer is capped to 1024 bytes (reception memory). Use the UDP streaming for faster transfers.

BUS TRANSFER EXAMPLES

Command string: `HARDWARE:SPY:UART`

Device transmission: `0`

hardware.uart

- SCPI Syntax: `HARDWARE:UART`
- SCPI Syntax: `HARDWARE:UART?`
- Shell Hook: `io`

Slow Blocking

Perform an IO operation over the UART bus. Query mean read, Set mean write

ARGUMENTS / RETURNS

Argument	Description
`[PAYLOAD]`	int(s),<0x00--0xFF>

Returns	Description
`STATUS`	int<0...>
`[PAYLOAD]`	int(s),<0x00--0xFF>

ADDITIONAL INFORMATION

> **WARNING**
>
> As this bus is asynchronous, the read command may send back a LOT of data, capped to 1024 bytes (reception memory). If speed is a requirement, consider using the raw UDP streaming.

BUS TRANSFER EXAMPLES

Command string: `HARDWARE:UART[?] [0x487643]`

Device transmission: `0,[0x7865456]`

Logging controls

logger.subscribe

- SCPI Syntax: `LOGGER:SUBSCRIBE`
- Shell Hook: `logger`

Very fast (<10µs) Non-Blocking

Connect a log source (spy, serial bus, Filesystem, UDP Stream ...) to a sink (Filesystem, UDP Stream...).

ARGUMENTS / RETURNS

Argument	Description
`SOURCE`	str<...>
`TARGET`	str<...>

Returns	Description
`STATUS`	int<0...>
`ID`	int<0...>

BUS TRANSFER EXAMPLES

Command string: `LOGGER:SUBSCRIBE /serial/usart0,/network/udp`

Device transmission: `0,19`

logger.unsubscribe

- SCPI Syntax: `LOGGER:UNSUBSCRIBE`

Very fast (<10µs) Non-Blocking

Disconnect a link.

ARGUMENTS / RETURNS

Argument	Description
`ID`	int<0...>

Returns	Description
`STATUS`	int<0...>

BUS TRANSFER EXAMPLES

Command string: `LOGGER:UNSUBSCRIBE 19`

Device transmission: `0`

Target Programming Interface

programmation.connect

- SCPI Syntax: `PROGRAMMATION:CONNECT`
- Shell Hook: `programmer`

Very slow Blocking

Connect the DUT over the SWD/JTAG buses.

ARGUMENTS / RETURNS

Argument	Description
`BUS`	str

Returns	Description
`STATUS`	int<0...>

ADDITIONAL INFORMATION

> **IMPORTANT**
>
> This command is built on CPU polled interfaces. No other operation will work when running it.

BUS TRANSFER EXAMPLES

Command string: `PROGRAMMER:CONNECT?`

Device transmission: `0`

programmation.detect

- SCPI Syntax: `PROGRAMMATION:DETECT`
- Shell Hook: `programmer`

Very slow Blocking

Detect if the DUT is present over the SWD/JTAG buses.

ARGUMENTS / RETURNS

Argument	Description
`BUS`	str

Returns	Description
`STATUS`	int<0...>
`RESULT`	int<0(yes),1(no)>

ADDITIONAL INFORMATION

> **IMPORTANT**
>
> This command is built on CPU polled interfaces. No other operation will work when running it.

BUS TRANSFER EXAMPLES

Command string: `PROGRAMMER:DETECT?`

Device transmission: `0,0`

programmation.device

- SCPI Syntax: `PROGRAMMATION:DEVICE`
- Shell Hook: `programmer`

Very slow Blocking

Read the DUT descriptions values over the SWD/JTAG buses.

ARGUMENTS / RETURNS

Argument	Description
`BUS`	str

Returns	Description
`STATUS`	int<0...>
`VALUES`	int[]<0x98765432,...>

ADDITIONAL INFORMATION

> **IMPORTANT**
>
> This command is built on CPU polled interfaces. No other operation will work when running it.

BUS TRANSFER EXAMPLES

Command string: `PROGRAMMER:DEVICE?`

Device transmission: `0,0x98765432`

programmation.flash

- SCPI Syntax: `PROGRAMMATION:FLASH`
- Shell Hook: `programmer`

Very slow Blocking

Flash the DUT over the SWD/JTAG buses.

ARGUMENTS / RETURNS

Argument	Description
`BUS`	str
`FILE`	str<...>

Returns	Description
`STATUS`	int<0...>

ADDITIONAL INFORMATION

> **IMPORTANT**
>
> This command is built on CPU polled interfaces. No other operation will work when running it.

BUS TRANSFER EXAMPLES

Command string: `PROGRAMMER:FLASH /sd/firmware.bin`

Device transmission: `0`

programmation.read

- SCPI Syntax: `PROGRAMMATION:READ`
- Shell Hook: `programmer`

Very slow Blocking

Read the DUT memory over the SWD/JTAG buses. Push the data over the standard logger source

ARGUMENTS / RETURNS

Argument	Description
`BUS`	str

Returns	Description
`STATUS`	int<0...>

ADDITIONAL INFORMATION

> **IMPORTANT**
>
> This command is built on CPU polled interfaces. No other operation will work when running it.

BUS TRANSFER EXAMPLES

Command string: `PROGRAMMER:READ?`

Device transmission: `0`

programmation.status

- SCPI Syntax: `PROGRAMMATION:STATUS`
- Shell Hook: `programmer`

Very slow Blocking

CHeck if the DUT is still running over the SWD/JTAG buses.

ARGUMENTS / RETURNS

Argument	Description
`BUS`	str

Returns	Description
`STATUS`	int<0...>
`RESULT`	int<0(yes), 1 (false)

ADDITIONAL INFORMATION

> **IMPORTANT**
>
> This command is built on CPU polled interfaces. No other operation will work when running it.

BUS TRANSFER EXAMPLES

Command string: `PROGRAMMER:STATUS?`

Device transmission: `0,0`

programmation.verify

- SCPI Syntax: `PROGRAMMATION:VERIFY`
- Shell Hook: `programmer`

Very slow Blocking

Verify the flashed binaries over the SWD/JTAG buses.

ARGUMENTS / RETURNS

Argument	Description
`BUS`	str

Returns	Description
`STATUS`	int<0...>
`RESULT`	int<0 (sucess),1 (error)>

ADDITIONAL INFORMATION

> **IMPORTANT**
>
> This command is built on CPU polled interfaces. No other operation will work when running it.

BUS TRANSFER EXAMPLES

Command string: `PROGRAMMER:VERIFY? /sd/firmware.bin`

Device transmission: `0,0`

Bus scanning

scan.can

- SCPI Syntax: `SCAN:CAN?`
- Shell Hook: `scan`

Very slow Blocking

Scan the CAN bus for any active device on it.

ARGUMENTS / RETURNS

This command does not take any arguments.

Returns	Description
`STATUS`	int<0...>
`RESULT`	int<0(yes),1(no)>

BUS TRANSFER EXAMPLES

Command string: `SCAN:CAN?`

Device transmission: `0,0`

scan.i2c

- SCPI Syntax: `SCAN:I2C?`
- Shell Hook: `scan`

Very slow Blocking

Scan the I2C bus for any active device on it.

ARGUMENTS / RETURNS

This command does not take any arguments.

Returns	Description
`STATUS`	int<0...>
`RESULT`	int<0(yes),1(no)>

BUS TRANSFER EXAMPLES

Command string: `SCAN:I2C?`

Device transmission: `0,0`

scan.spi

- SCPI Syntax: `SCAN:SPI?`
- Shell Hook: `scan`

Very slow Blocking

Scan the SPI bus for any active device on it.

ARGUMENTS / RETURNS

This command does not take any arguments.

Returns	Description
`STATUS`	int<0...>
`RESULT`	int<0(yes),1(no)>

BUS TRANSFER EXAMPLES

Command string: `SCAN:SPI?`

Device transmission: `0,0`

scan.uart

- SCPI Syntax: `SCAN:UART?`
- Shell Hook: `scan`

Very slow Blocking

Scan the UART bus for any active device on it.

ARGUMENTS / RETURNS

This command does not take any arguments.

Returns	Description
`STATUS`	int<0...>
`RESULT`	int<0(yes),1(no)>
`[BUS]`	int<0,1,2>

BUS TRANSFER EXAMPLES

Command string: `SCAN:UART?`

Device transmission: `0,0,1`

Sequencing

sequences.add

- SCPI Syntax: `SEQUENCES:ADD`

Fast (<100µs) Non-Blocking

Add an instruction on the sequence. Instruction will be 'compiled'.

ARGUMENTS / RETURNS

Argument	Description
`NAME`	str<...>
`INSTRUCTION`	str<...>

Returns	Description
`STATUS`	int<0...>

ADDITIONAL INFORMATION

> **WARNING**
>
> For complex sequences, consider transfering it as a whole over the FTP server.

BUS TRANSFER EXAMPLES

Command string: `SEQUENCE:ADD flasher execute *IDN`

Device transmission: `0`

sequences.get

- SCPI Syntax: `SEQUENCES:GET?`

Slow Blocking

Send the binary file that correspond to the sequence

ARGUMENTS / RETURNS

Argument	Description
`NAME`	str<...>

Returns	Description
`STATUS`	int<0...>
`DATA`	<int[]<0x98,0x76...>

BUS TRANSFER EXAMPLES

Command string: `SEQUENCE:GET`

Device transmission: `0,0x98,0x87...`

sequences.new

- SCPI Syntax: `SEQUENCES:NEW`

Fast (<100µs) Non-Blocking

Add a new sequence on the device

ARGUMENTS / RETURNS

Argument	Description
`NAME`	str<...>

Returns	Description
`STATUS`	int<0...>

BUS TRANSFER EXAMPLES

Command string: `SEQUENCE:NEW flasher`

Device transmission: `0`

sequences.run

- SCPI Syntax: `SEQUENCES:RUN`

Slow Non-Blocking

Execute the sequence.

ARGUMENTS / RETURNS

Argument	Description
`NAME`	str<...>

Returns	Description
`STATUS`	int<0...>
`PASS`	int<0-...>
`FAIL`	int<0-...>

BUS TRANSFER EXAMPLES

Command string: `SEQUENCE:RUN flasher`

Device transmission: `0,1,0`

IEEE-488.2 Standard Core Commands

*cls

- SCPI Syntax: `*CLS`

Very fast (<10µs) Blocking

Clear all the SCPI exposed registers on the device.

ARGUMENTS / RETURNS

This command does not take any arguments.

Returns	Description
`STATUS`	int<0...>

ADDITIONAL INFORMATION

> **WARNING**
>
> This does not perform a reset of the device. Hence, statuses may be retained.

BUS TRANSFER EXAMPLES

Command string: `*CLS`

Device transmission: `0`

***ese**

- SCPI Syntax: `*ESE`
- SCPI Syntax: `*ESE?`

Very fast (<10µs) Blocking

Set bits on the Event Status Register. Returns the event status mask.

ARGUMENTS / RETURNS

Argument	Description
`XX`	Bits to be set (if maskable).

Returns	Description
`STATUS`	int<0...>
`MASK`	int<-1-255>

ADDITIONAL INFORMATION

Bit signification

Bit	Name	Signification
0	OPC	Operation complete
1		Unused
2	QYE	The device could not process a command. Buffers are perhaps full ?
3	DDE	Self test failed
4	EXE	A command failed it's execution
5	CMD	An invalid command was received
6		Unused
7		Unused

BUS TRANSFER EXAMPLES

Command string: `*ESE 255`

Device transmission: `0,255`

---

*idn

- SCPI Syntax: `*IDN?`

Very fast (<10µs) Blocking

Return the identification string for the device.

ARGUMENTS / RETURNS

This command does not take any arguments.

Returns	Description
`STATUS`	int<0...>
`IDN`	

BUS TRANSFER EXAMPLES

Command string: `*IDN?`

Device transmission: `0,EtherBenchv1:[build-date]:[firmware-version]`

***opc**

- SCPI Syntax: `*OPC`
- SCPI Syntax: `*OPC?`

Very fast (<10µs) Blocking

Check for current running commands

ARGUMENTS / RETURNS

Argument	Description
`XX`	Value

Returns	Description
`STATUS`	int<0...>

BUS TRANSFER EXAMPLES

Command string: `*OPC?`

Device transmission: `0,OK`

*opt

- SCPI Syntax: `*OPT`

Very fast (<10µs) Blocking

Returns the options string of the device.

ARGUMENTS / RETURNS

This command does not take any arguments.

Returns	Description
`STATUS`	int<0...>

BUS TRANSFER EXAMPLES

Command string: `*OPT?`

Device transmission: `0,EtherBenchv1:[]`

*rst

- SCPI Syntax: `*RST`

Slow Blocking

Reset the device, equivalent to a power on-off-on cycle.

ARGUMENTS / RETURNS

This command does not take any arguments.

Returns	Description
`STATUS`	int<0...>

ADDITIONAL INFORMATION

WARNING

Perform an hard reset of the device. Any openned sessions will be *brutally* closed.

BUS TRANSFER EXAMPLES

Command string: `*RST`

Device transmission: `0`

***sre**

- SCPI Syntax: `*SRE`
- SCPI Syntax: `*SRE?`

Very fast (<10µs) Blocking

Return the service query status.

ARGUMENTS / RETURNS

This command does not take any arguments.

Returns	Description
`STATUS`	int<0...>

ADDITIONAL INFORMATION

> **NOTE**
>
> This command is implented as a placeholder. Always return false.

BUS TRANSFER EXAMPLES

Command string: `*SRE?`

Device transmission: `0`

*stb

- SCPI Syntax: `*STB?`

Very fast (<10µs) Blocking

Read the status byte register

ARGUMENTS / RETURNS

This command does not take any arguments.

Returns	Description
`STATUS`	int<0...>

ADDITIONAL INFORMATION

> **NOTE**
>
> This command is implented as a placeholder. Always return 0.

BUS TRANSFER EXAMPLES

Command string: `*STD?`

Device transmission: `0`

*tst

- SCPI Syntax: `*TST?`

Very fast (<10µs) Blocking

Perform the self-test of the device. Check that all threads are running.

ARGUMENTS / RETURNS

This command does not take any arguments.

Returns	Description
`STATUS`	int<0...>

BUS TRANSFER EXAMPLES

Command string: `*TST?`

Device transmission: `0`

---

*wai

- SCPI Syntax: `*WAI`

Slow Blocking

Wait for all operations to be executed.

ARGUMENTS / RETURNS

This command does not take any arguments.

Returns	Description
`STATUS`	int<0...>

BUS TRANSFER EXAMPLES

Command string: `*WAI`

Device transmission: `0`

POSIX Filesystem & Shell Commands

cat

- Shell Hook: `cat`

Normal Blocking

Show a file from the filesystem into the current terminal

ARGUMENTS / RETURNS

Argument	Description
`file`	str

Returns	Description
`CAT`	str

BUS TRANSFER EXAMPLES

Command string: `cat /sd/hello.txt`

Device transmission: `Hello World !`

cd

- Shell Hook: `cd`

Fast (<100µs) Blocking

Change the current directory

ARGUMENTS / RETURNS

Argument	Description
`target`	str

Returns	Description
`CD`	OK

BUS TRANSFER EXAMPLES

Command string: `cd /sd`

Device transmission: ``

---

clear

- Shell Hook: `clear`

Very fast (<10µs) Blocking

Clear the current terminal

ARGUMENTS / RETURNS

This command does not take any arguments.

Returns	Description
`RST`	OK

BUS TRANSFER EXAMPLES

Command string: `clear`

Device transmission: ``

ls

- Shell Hook: `ls`

Normal Blocking

List files on the current directory

ARGUMENTS / RETURNS

This command does not take any arguments.

Returns	Description
`LS`	...

BUS TRANSFER EXAMPLES

Command string: `ls`

Device transmission: `hello.txt`

---

mkdir

- Shell Hook: `mkdir`

Fast (<100µs) Blocking

Create a directory on the current location

ARGUMENTS / RETURNS

Argument	Description
`NAME`	str

Returns	Description
`MKDIR`	OK

BUS TRANSFER EXAMPLES

Command string: `mkdir /sd/test/`

Device transmission: ``

mv

- Shell Hook: `mv`

Slow Blocking

Move a file from a location to another one

ARGUMENTS / RETURNS

Argument	Description
`SOURCE`	str
`TARGET`	str

Returns	Description
`MV`	OK

BUS TRANSFER EXAMPLES

Command string: `mv /sd/hello.txt /sd/hello2.txt`

Device transmission: ``

pwd

- Shell Hook: `pwd`

Normal **Blocking**

Print the current location on the filesystem

ARGUMENTS / RETURNS

This command does not take any arguments.

Returns	Description
`PWD`	str

BUS TRANSFER EXAMPLES

Command string: `PWD`

Device transmission: `/sd/example/`

Examples

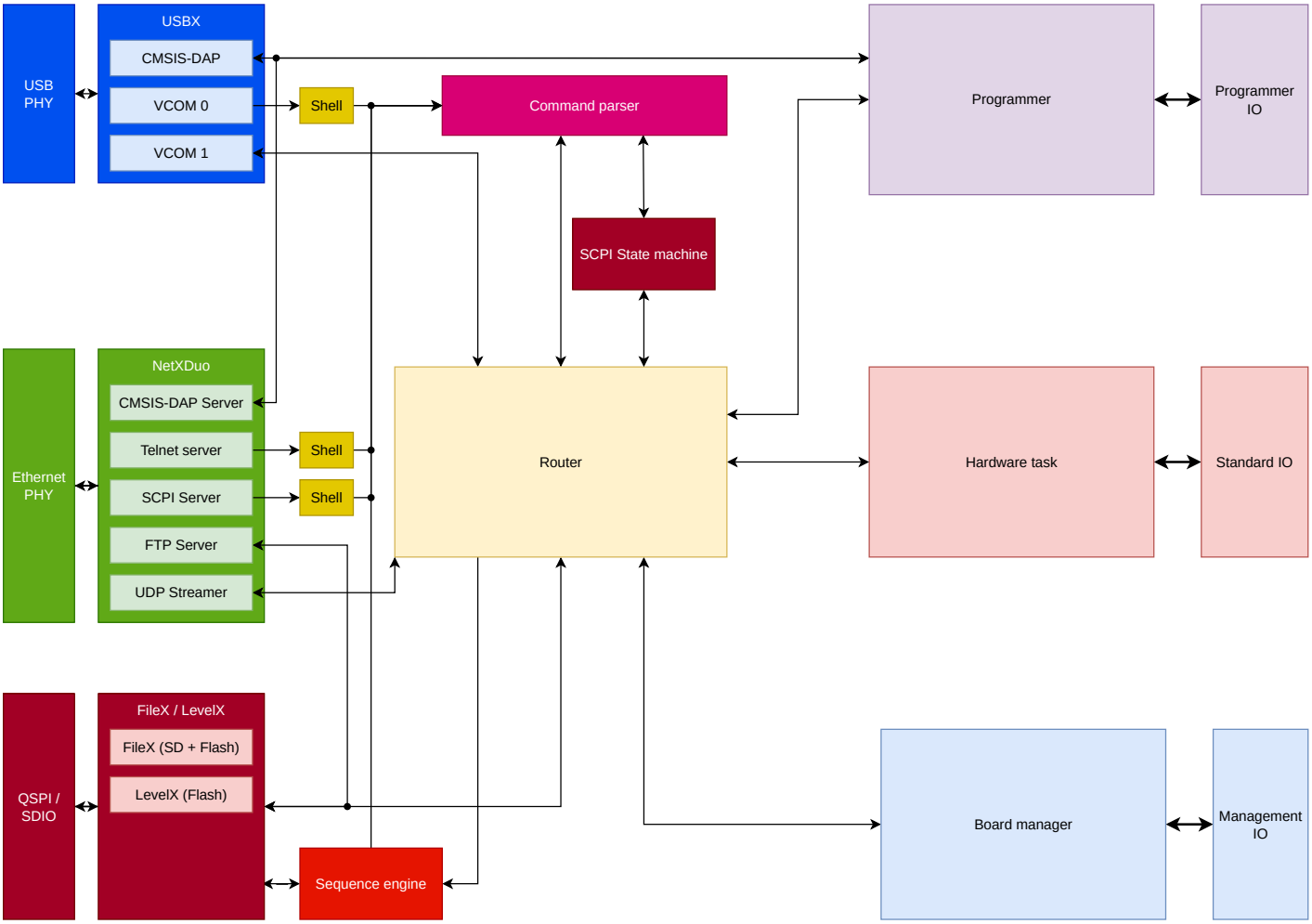
Architecture

For this last parts, the documentation address more to developper, or just hobbyist which want to understand how the device work. This chapter is far from being mandatory, in fact, I do not recommended reading it if you're only willing to use the device as a tool.

In this chapter, topics like the software stack, the services we're running and advanced tricks to turn standard peripherals into a SWD / JTAG master will be described.

Global architecture

On the software side, the global software architecture is done on different threads, using the Eclipse ThreadX RTOS as a base.



All of the different threads are going to be described on their own pages. The remaining

Board manager

The board manager process is one of most important thread to be runned, as it rules all behaviors of the board, without the user knowing it. Therefore, under it's own responsability are placed :

- The I2C screen
- The USB Hub
- The DUT regulator
- The daughter board EEPROM
- The temperature sensors

That's done because all of them share a single bus, the I2C peripheral. Thus, we don't need heavy mutex for each access.

To remains coherent within the architecture paradigm we choosed, this thread remains quite independant. Except requests for screen messages, and DUT voltage, it's isolated from the other.

Life cycle

This thread is among the first to be started. Once launched, it'll start initializing all the peripheral, essentially the USB Hub. Then, it'll loop at a frequency of 100 Hz to adjust the voltage of the DUT and monitor values.

This thread can't be stopped nor paused.

Battery emulation mode

If enabled, the DUT voltage will follow the following equation, derived from the Sheperd equation. The ony different is the lack of the $R \cdot I$ term, which can't be properly modeled with a discrete controller. It corresponds to the ESR value of the resistance, the user must place it physically, in series with the power supply.

$$V_{batt} = E_0 - K \cdot \left(\frac{Q}{Q - it} \right) \cdot it + A \cdot e^{-B \cdot it}$$

Cmsis eth

Cmsis usb

Command parser

Hardware IO

This task has one primary responsibility: managing all serial I/O from and to the DUT (Device Under Test). This thread is arguably one of the most complex in the system. It must ensure high-speed, non-blocking writes while simultaneously operating in slave or "spy" mode to capture asynchronous incoming data and stream it to the appropriate target.

To meet these strict timing constraints without overwhelming the CPU, the task heavily relies on Direct Memory Access (DMA) for both transmission and reception. Rather than using costly byte-by-byte interrupts or inefficient fixed-interval polling, it leverages STM32 hardware-level frame detection to flush buffers and route packets dynamically.

Responses are broadcasted using a configurable address mask. This publish-subscribe approach ensures that the hardware task can simultaneously route data to the terminal AND to a physical log file (e.g., via FileX).

RTOS Behavior

The Hardware IO task remains in a suspended state, waiting for an Event Flag (rather than a simple semaphore) to resume execution. This wake-up event can be triggered by two primary sources:

- The main Router dispatching a new datagram to the task's input queue.
- Any managed peripheral triggering a hardware interrupt (e.g., Transfer Complete or Idle line detection).

Upon waking up, the thread evaluates the event flags to instantly identify the source and executes the corresponding state machine logic without blocking.

Peripherals

This thread manages a comprehensive suite of hardware interfaces, including:

- 1x CAN bus
- 3x USART
- 1x I2C
- 1x SPI
- 2x DAC
- 2x ADC

All of these are fully available to the user for arbitrary operations. The user can seamlessly transition these digital peripherals into a "Slave" mode. In this configuration, they act as a passive bus spy: sniffing and decoding the data without ever driving the bus lines.

This operational mode switch can be performed on the fly by sending the appropriate control command. There is no software restriction regarding the mode assignment of digital peripherals. However, the analog section remains locked to its native state by design.

Master Mode

When operating in master mode, all transactions are initiated by the user. The task optimizes the transfer methodology based on the payload size:

- **Polling/IT** for very short transfers (minimizing DMA setup overhead).
- **DMA** for larger payloads.

This dynamic optimization is entirely transparent to the user, as the software interface remains identical.

> ⚠ **Master-Slave Electrical Conflicts**
>
> Forcing a peripheral into Master mode while externally driven is prohibited and electrically hazardous. In Master mode, the output lines are actively driven by the MCU. Attempting to force an external voltage on an actively driven output pin can result in permanent hardware damage. However, protective I/O buffers are implemented on the daughter boards to mitigate these risks.

Responses are generated and dispatched as soon as a hardware transfer completes. To prevent task starvation and ensure high responsiveness, the thread employs an asynchronous, non-blocking state machine. If a new command targets a peripheral that is currently busy executing a previous transfer, the request is pushed at the end of it's input queue, allowing the thread to continue servicing other active peripherals.

Slave Mode

When configured in slave mode, the peripheral reacts to external stimuli identically to a standard sensor. This is highly useful for sniffing and decoding bus traffic using the MCU's native hardware parsers, providing an extremely efficient tool for advanced debugging.

In slave mode, data packets are flushed and dispatched upon detecting the end of a transaction, leveraging hardware events:

- **USART:** Idle line detection (RX line high for a full frame).
- **I2C:** STOP condition detected on the bus.
- **SPI:** Rising edge detected on the Chip Select (SSx) pin.
- **CAN:** Complete frame successfully received in the hardware FIFO.

Once captured, these payloads are wrapped into the standardized IPC message format and forwarded to the Router for proper dispatching based on the active address mask.

Programmer

To emulate the SWD or JTAG signals, we are using a pair of SPI peripherals whose pins can be multiplexed to USART buses. This hardware topology allows for the complete emulation of the ARM Specification.

Peripherals

The peripheral configuration is mapped as follows:

Mode	`SPI2`	`SPI3`	`USARTx`
SWD		**X**	Optional*
JTAG	**X**	**X**	

> **USART Usage**
>
> As permitted by the SWD specification, an additional pin (**SWO**) can be used for high-speed trace logging. This asynchronous pin can be directly routed to a raw USART peripheral, justifying its potential allocation in this mode.

The `SPI3` peripheral is strictly configured as a Master to generate the reference clock (TCK). The `SPI2` peripheral operates exclusively in JTAG mode as a Slave. On the PCB layout, the `SPI3` clock pin is physically tied to the `SPI2` clock pin. This hardware constraint must be strictly respected during firmware development.

The `SPI3` is configured as half duplex, due to the direction changes. The `SPI2` remains as full duplex to handle the JTAG.

Pin	Source Peripheral
`TCK` / `SWCLK`	`SPI3_SCLK`
`TMS` / `SWDIO`	`SPI3_IO`
`TDI`	`SPI2_MOSI`
`TDO`	`SPI2_MISO`
`RST`	`GPIO`
`PRESENT`	`GPIO`

Consequently, for any transfer to occur on `SPI3`, an active transfer **MUST** be simultaneously running on `SPI2`. Furthermore, data must be preloaded into the `SPI3` buffers before initiating the master clock.

Exploitation

The SWD protocol can be particularly tedious to implement since its transfer widths are not byte-aligned. Leveraging the latest generation of ST peripherals significantly mitigates this issue, as the hardware transfer size can be dynamically adjusted between 4 and 32 bits. We chose a default transfer size of 16 bits. This ensures reliable data shifting while providing sufficient margin to adapt the transfer size without violating the protocol timings. A dedicated algorithm computes the required number of 16-bit transfers and dynamically adjusts the frame size for the final residual bits.

Conversely, the JTAG protocol is more straightforward to implement. It relies on continuous, larger frames that can be processed as large byte arrays shifted directly by the DMA.

In all cases, due to the critical timings and turnaround quirks inherent to these protocols, the programmer engine must run as a dedicated, high-priority task. It may rely on polling for ultra-short transfers. As a result, lower-priority tasks might experience slightly increased latencies while a programming sequence is actively running.

RTOS Behavior

As highlighted above, the programmer holds a critical position within the global architecture. To guarantee strict signal timings, the programmer task is authorized to dynamically disable RTOS preemption for specific, time-critical operations. However, this lock is never maintained during long, DMA-backed transfers.

Because programming latencies are generally tighter than standard bus operations, the payload data does not flow through the main Router. Instead, the CMSIS-DAP front-end server communicates directly with the programmer queues.

Other Operation Modes

From a broader perspective, the programmer block is optimized for large data transfers, utilizing its own dedicated memory pool and queues. This architecture allows it to handle more generic operations natively—such as direct EEPROM read/write sequences—without requiring the execution of a custom script. Should a specific I/O profile be unsupported natively, it will fallback to the standard `Hardware IO` task, trading off latency for flexibility.

Accessing Other Peripherals

As depicted on the master architectural schematic, the programmer *can* interface with the main Router and the `Hardware IO` task. This cross-linking ensures future scalability, enabling the support of protocols that cannot be handled by the SPI pair alone. A prime example is the ICSP protocol (AVR), which may require custom bit-banging sequences or specific target voltages.

Router

The router task is one of the most important one on the design. It's job is to push, in the right FIFO the incomming message. It handle duplication of the data, as a message can have different destinations. In that case garbage collection is handled.

All messages are send on the following structure :

```
typedef struct router_messge_t {
    ROUTER_SRC src;
    ROUTER_DEST dest;
    ROUTER_TYPE type;
    ROUTER_STATUS status;
    uint16_t size;
    ROUTER_PAYLOAD payload;
    ...
};
```

The different types are all enums, defined in the corresponding header file. The payload can be included in the header if small enough (< 16 bytes), or can be allocated on a TX_POOL, which is the recommended method.

Additionnaly, a response pointer function is passed, to ensure the command parser can respond while using the right tool, that correspond to the source. This make the whole system agnostic to the source of the command.

Scpi

Sequences

Shell handler

This process, without being one to be properly speaking is a tool that has one job : Identify commands, and sanitize the input stream. It'll send the command once a `\n` char has been found. This free some ressources for the command parser.

This utility is directly runned by the calling procedure, which can be both sides.